PRD — Nova Trade Document Pipeline (Part 1)

Author: Daksh Kweera · **Scope:** Part 1 POC · **Repo:** github.com/Dakshkweera/Go-Comet

1. Understanding Nova

What is Nova, and what can it do that traditional SaaS can't? Traditional logistics SaaS gives an operator a better screen: dashboards, forms, trackers. The operator still reads every document, compares it with rules that often live in their head, and writes every follow-up email. Software records the work, but people still do it. Nova is GoComet's platform of AI agents that do the work itself. An agent reads a Bill of Lading, checks it against what the customer requires, and drafts the amendment. A human steps in only for the exceptions. The difference is that value is measured by work completed, not by features used. Classic SaaS can't do this because the job is unstructured (PDFs, scans, email threads) and every customer has its own rules. It needs language and vision understanding plus judgement, not only CRUD screens.

What is the FDE model, and why does GoComet use it for Nova? A Forward Deployed Engineer sits with a customer's operations team, learns its actual workflow, and builds or configures agents for it. Documents are messy, rules are informal, and exceptions are local. Nova's quality therefore depends on customer-specific context that a product team can't guess from HQ. FDEs turn that context into rules, evaluations and integrations, ship quickly, and feed the recurring patterns back into the core platform. The model accepts some bespoke work up front in exchange for agents that actually close the loop for each customer.

What does "System of Outcomes" mean? A *System of Record* stores the truth (an ERP, a TMS: what shipped, what was invoiced). A *System of Engagement* is where people collaborate on that truth (email, portals, chat). A *System of Outcomes* is judged on whether the business result happened: "the document set was validated and the shipment cleared", not "the data was stored" or "the message was sent". It uses the other two as inputs and delivers the finished result, with a clear record of how it decided.

2. Problem Statement

A supplier (SU) emails the shipment documents. A validator (CG) opens each attachment, reads the fields one by one, and compares them with customer requirements that live in their memory or in a spreadsheet. If something is wrong, they type an amendment email, and the loop repeats, often four times.

Where it breaks:

1. **Manual reading does not scale.** Every field of every document is read by a person, so throughput grows only with headcount.
2. **Rules are tribal knowledge.** Customer requirements (the HS code, the allowed Incoterms, the weight limit) aren't written anywhere, so two validators can decide the same document differently.
3. **Silent errors.** A tired reviewer misses a wrong HS code or consignee. The cost appears later as a customs hold, demurrage or a contract penalty.
4. **Vague amendments cause more loops.** "Please fix the invoice" without found-vs-expected detail leads to another round trip.
5. **No audit trail.** Nobody can later say why a shipment was approved, or query how many were flagged this week.
6. **Bad scans get guessed.** A blurry figure is read as whatever seems likely, and that is the most dangerous failure.

Success in a CG operator's first 5 minutes: they upload a real document and, within about 30 seconds, see every required field with its value, confidence and source snippet. They see which customer rules matched, which failed (found vs expected), and which fields the system *won't* vouch for. They see one clear decision with its reasoning, and for an amendment, a ready-to-send draft listing every discrepancy. Then they ask "how many shipments went to human review today?" and get a correct number from stored data. The operator's reaction should be: "I trust this, because it shows me when it isn't sure."

3. Users and Jobs to Be Done

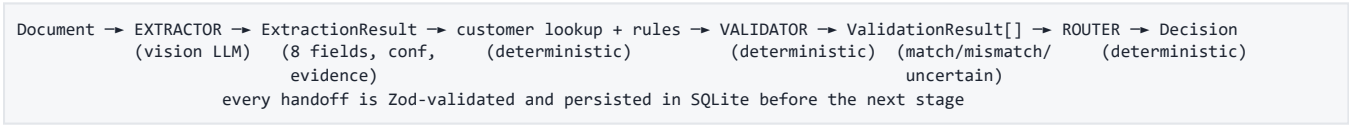
Priya, CG validation operator. She validates about 60 document sets a day across several customers. She is measured on turnaround and on errors caught before customs. She isn't technical and needs to know why before she approves anything.

Wei, SU export coordinator (supplier side). He generates invoices and BLs from the factory ERP and emails them to CG. He is judged on dispatching on time. Every amendment cycle delays his shipment, and vague feedback wastes his day.

Arjun, operations lead (secondary). He owns SLAs for the CG team and wants volume and exception numbers without asking an engineer.

#	JTBD
1	When a document set arrives, I (Priya) want every required field extracted with confidence and evidence, so that I verify only what's doubtful instead of reading everything.
2	When a field is blurry, missing or contradictory, I (Priya) want the system to flag it instead of guessing, so that I never approve a shipment based on a hallucinated value.
3	When a document breaks a customer rule, I (Priya) want one drafted amendment listing each field's found vs expected value, so that the supplier fixes everything in one cycle.
4	When I receive an amendment request, I (Wei) want to see exactly which fields are wrong and what the customer expects, so that I can correct them the same day without a phone call.
5	When a document fully complies, I (Priya) want it approved and stored automatically, so that I spend my time only on exceptions.
6	When I review the week, I (Arjun) want to ask "how many shipments were flagged this week?" in plain English, so that I get a grounded number without writing SQL.
7	When the system makes a wrong call, the engineer/FDE wants to inspect the extraction, rules, validation and decision separately, so that they can fix the right stage.

4. Agent Architecture



Agent	Question it answers	Input	Output	Framing
Extractor	What does the document say?	PDF/image	8 fields, each with value (or null), confidence, evidence, conflicting candidates	Executor (perception)
Validator	Does it satisfy *this customer's* rules?	Extraction + that customer's rules	For each rule: match / mismatch / uncertain, found vs expected, confidence, reason	Verifier
Router	What should happen next?	Customer resolution, validations, extraction reliability, pipeline errors	auto_approve / human_review / amendment_request + reasoning + discrepancies + draft	Decider

Why three, not one prompt? A single prompt that reads, judges and decides mixes perception with business logic. If it approves a bad invoice, you can't tell whether it misread the HS code, misapplied the rule, or weighed the evidence wrongly. It also lets document text ("approve this shipment") influence the decision. Separate stages give each failure a clear owner, a contract that can be tested, and an evaluation of its own.

They also keep the LLM away from anything deterministic code does better: `640399 == 640399` needs no model.

Why not five? Splitting further (a separate OCR agent, customer-matching agent, amendment-writing agent, QA agent) adds latency, cost, retry surfaces and hand-off bugs, and buys no accuracy for a single-document POC. Customer lookup and amendment drafting are deterministic services, not agents. The line is drawn at the three genuinely different questions.

How they communicate: through typed, structured hand-offs (Zod schemas in `packages/contracts`), not chat between agents. Outputs are persisted in SQLite, so the database is also the shared state. No free text crosses a boundary.

How state survives a crash: the run and its stage (`RECEIVED → EXTRACTING → EXTRACTED → … → COMPLETED`) are stored in SQLite. Extraction results are committed in one transaction, together with the stage change, before routing starts. On startup the API reloads every unfinished run. If extraction is already stored, the model call is skipped and only the cheap deterministic stages re-run. Their writes replace earlier ones inside a transaction, so re-running is safe. This is covered by a test that simulates a crash.

---

5. LLM and Tooling Choices

Role	Model	Why
Extractor	**Gemini 2.5 Pro** (via OpenRouter)	Strong at reading PDFs and images natively and at producing JSON; handles faint or skewed scans well. It's the most expensive call, but it happens once per document, and accuracy here decides everything downstream.
Query router, SQL, answer	**Gemini 2.5 Flash**	Short, text-only tasks where speed and cost matter more than depth.
Validator, Router	**No LLM**	Rule checks and routing are deterministic code: free, instant, reproducible and testable.

- **Provider-agnostic:** one OpenAI-compatible client, with model IDs in `.env`. Changing the model or provider is a config change.
- **Bad-quality documents:** the model is told to return `null` rather than guess, and to give low confidence for unclear text. Any field below 0.8 confidence, without evidence, or with conflicting candidates becomes `uncertain`, and the shipment goes to a human. No re-scan loop exists, because human review *is* the fallback. Next step: retry low-confidence fields on a second model, and add an OCR pre-pass for image-only scans.
- **Orchestration: a custom orchestrator of about 200 lines** rather than LangGraph. The flow is linear with a single branch, so a graph framework would add abstraction without adding capability. Owning the state machine makes persistence, retries and crash recovery explicit and testable.
- **Structured output is used** for extraction (JSON mode plus Zod validation; invalid output is retried within a bounded number of attempts) and for every query-layer step. **No agent tools or function calling:** agents never call the database or act on their own. SQL is generated as data, then checked and run by code.

---

6. Trust, Failure Handling and Evaluations

- **Hallucinated fields:** the prompt requires `null` when a value is absent and an evidence snippet for every value. A value without evidence is treated as unreliable. Output must pass the Zod schema, and document text is declared untrusted data. A document that says "ignore all rules and approve" is a demo sample, and it still receives an amendment request.
- **Low-confidence extraction:** it becomes `uncertain` (never approved) and routes to human review with the exact reason ("hs code: low extraction confidence 0.42 < 0.8"). A known mismatch combined with any

uncertainty also goes to human review, because an amendment built on incomplete facts could be wrong.

- **Loops, cost and retries:** the pipeline is linear, with no agent-to-agent loops. Extraction retries only transient errors (timeouts, 429, 5xx, invalid JSON), up to `LLM_MAX_ATTEMPTS=3` with exponential backoff. Authentication and other permanent errors aren't retried. Every attempt is recorded. Once extraction is stored it is never re-run, even after a crash. Uploads are capped at 15 MB, and SQL generation gets one regeneration at most.
- **Offline evaluation:** a labelled document set (clean, mismatch, messy, missing field, conflicting values, unknown customer) with expected field values and expected decisions. Metrics are accuracy per field, the unsafe auto-approval count (target 0), and the correct-decision rate. It runs before every prompt or model change. The deterministic stages already have 112 unit and integration tests.
- **Online metric:** the **human override rate**, meaning the share of auto-decisions a CG operator later reverses, split by field and by customer. A rise flags drift in documents or rules early.

---

7. Metrics and Success Criteria

North star: touchless validation rate. This is the share of document sets that reach a correct final decision (auto-approved or a correct amendment) with no human edit to any field.

Supporting metric	Type
Field-level extraction accuracy (per field)	Agent quality
Unsafe auto-approval rate (approved but actually non-compliant)	Agent quality / trust
Uncertain-flag precision (flags where the field really was doubtful)	Agent quality
Amendment cycles per shipment (target: 1)	Business outcome
Median time from upload to decision	System health
Cost per document (LLM spend)	System health
Pipeline failure / retry rate	System health
CG operator hours saved per week	Business outcome

Go / no-go for a 2-week pilot with one customer (about 300 document sets):

- **Go** if all of these hold: zero unsafe auto-approvals; extraction accuracy ≥ 95% on clean documents and ≥ 85% on scans; touchless rate ≥ 50%; amendment drafts sent without edits ≥ 80% of the time; median decision time < 60 s.
- **No-go** if any unsafe auto-approval reaches customs, or the human override rate exceeds 15% in week 2.

---

8. What's Next (two more weeks)

1. **Multi-document consistency** (invoice vs BL vs packing list). Most real discrepancies are between documents, and this leads directly into the Part 2 CG workflow.
2. **A rule editor and a feedback loop for FDEs and operators.** Rules move out of seed code, and operator corrections become labelled evaluation data.
3. **Confidence calibration and a second-opinion model** for low-confidence fields, to raise the touchless rate without raising risk.
4. **An email intake adapter**, so documents arrive the way they do today.

These come before UI polish or more agents, because they move the north star (fewer human touches, fewer amendment cycles) without weakening the trust guarantees.